Threat Model and Secure Configuration

Project Report • Security Analysis of a Sample Web Application

This project identifies security risks in a sample web application that supports user registration, login, and profile viewing. It documents assets, data flows, trust boundaries, STRIDE threats, risk priorities, and recommended defensive controls. The scores below are illustrative and should be updated after reviewing the actual application.

1. Objectives

• Identify assets, users, data flows, and trust boundaries.

• Identify likely threats using STRIDE.

• Rank risks by likelihood and impact.

• Recommend proportionate defensive controls.

• Provide a hardening checklist and verification plan.

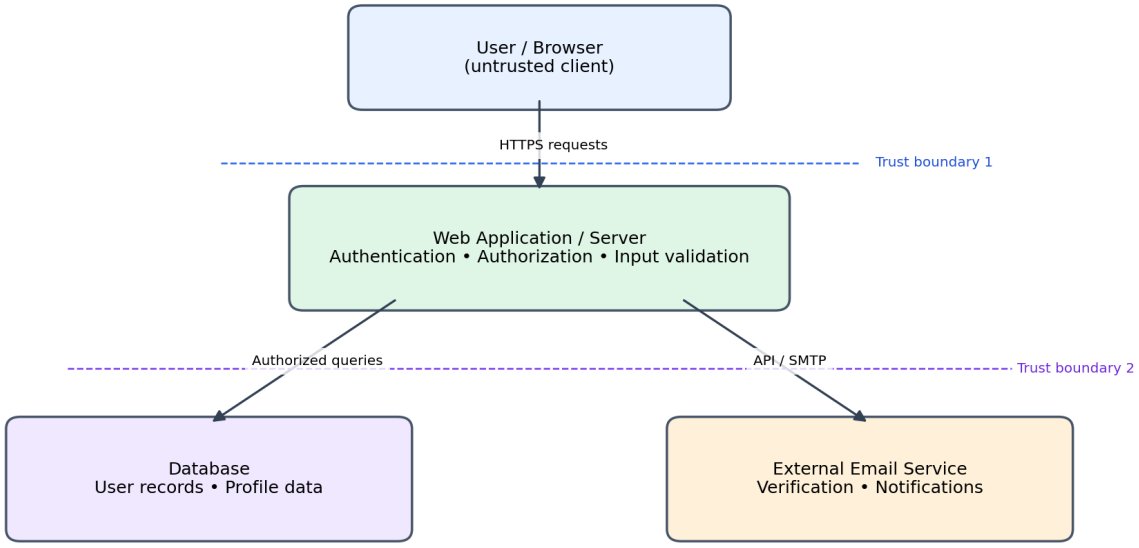
2. Scope and assumptions

The sample application supports user registration, login, and profile viewing. It uses a browser, a web application server, a database, and an external email service. This architecture is illustrative and must be adapted to the actual system.

3. Assets to protect

Asset	Description
User accounts	Authentication and account access
Passwords	Authentication secrets; store only secure password hashes
Personal data	User profile and other private information
Session tokens	Maintain authenticated sessions
Application server	Processes requests and enforces security rules
Database	Stores user records and application data
Application code and secrets	Business logic, configuration, API keys
Audit logs and backups	Support investigation and recovery

4. Threat-model diagram



Threat-model diagram — illustrative architecture; adapt to the actual application.

Trust boundary 1 separates the browser from the server. Trust boundary 2 represents controlled interactions between the server and data/external-service components.

5. STRIDE methodology

Category	Meaning / example
Spoofing	Pretending to be another user; e.g., use of stolen credentials.
Tampering	Unauthorized modification of requests or stored data.
Repudiation	Denying an action when audit evidence is insufficient.
Information Disclosure	Unauthorized exposure of private data or secrets.
Denial of Service	Making the service unavailable through excessive requests.
Elevation of Privilege	Gaining permissions beyond the user's role.

6. Risk register

Risk score = Likelihood × Impact. Both use a 1–5 scale. Suggested bands: 1–4 Low, 5–9 Medium, 10–15 High, 16–25 Critical.

ID / Threat	Asset	L	I	Score	Priority
T1 Account compromise / spoofing	User accounts	4	5	20	Critical
T2 SQL injection / tampering	Database	3	5	15	High
T3 Broken access control	Personal data	4	5	20	Critical
T4 Session theft or fixation	Session tokens	3	4	12	High
T5 Sensitive data exposure	Personal data and secrets	3	5	15	High
T6 Denial of service	Application server	3	4	12	High
T7 Insufficient logging / repudiation	Audit logs	3	3	9	Medium
T8 Privilege escalation	Admin functions	3	5	15	High

Scores are illustrative estimates for the sample system, not evidence from a live security test.

7. Prioritized hardening plan

Authentication and access: Enforce secure password hashing; add login rate limits; use MFA for privileged accounts where possible; verify authorization on every protected action; apply least privilege.

Application security: Use parameterized SQL queries; validate inputs server-side; encode output appropriately; use HTTPS; protect state-changing requests against CSRF where applicable.

Session security: Set HttpOnly, Secure, and appropriate SameSite cookie attributes; rotate session IDs after authentication; set idle/absolute expiry; invalidate sessions at logout.

Data and configuration: Keep secrets out of source code; use environment or secret-management configuration; disable production debug mode; return generic errors; restrict database network access.

Monitoring and recovery: Record relevant authentication and administrative events; restrict log access; configure backups; test restoration; keep dependencies patched; monitor unusual request volumes.

8. Verification test plan

Test	Expected secure behavior
Invalid login	Request is rejected without revealing whether an account exists; attempts are rate-limited.
Other user's profile	Server denies access to records the signed-in user is not authorized to view.
Unexpected input	Input is safely handled; no database command or script execution occurs.
Repeated requests	Rate limiting or other protective controls engage without exposing sensitive details.
Normal user opens admin page	Server returns an access-denied response.
Application error	Response does not expose stack traces, secrets, or internal configuration.
Security event review	Relevant events are recorded; passwords, tokens, and unnecessary personal data are not logged.

9. Completion steps

1. Identify application components, assets, users, and external dependencies.

2. Draw the data-flow diagram and mark trust boundaries.

3. Identify threats using STRIDE: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.

4. Score each threat using Likelihood × Impact on a 1–5 scale.

5. Address critical and high risks first; assign an owner and target date in a real project.

6. Implement and verify defensive controls in an authorized test environment.

7. Record evidence, remaining risks, and follow-up actions.

10. Expected results

• Documented assets, components, data flows, and trust boundaries.

• A STRIDE-based threat list and risk register.

• A prioritized security hardening plan.

• A checklist and test plan for verifying controls.

11. Conclusion

Threat modeling helps teams find and prioritize security risks before they become incidents. By documenting assets, applying STRIDE, scoring risks, and verifying suitable defensive controls, a development team can improve the security of its web application. The model must be reviewed whenever the application architecture or threat landscape changes.

Submission note

This package is a sample threat-modeling deliverable. The controls have not been independently implemented or tested. Upload the report and supporting files to one cloud folder or repository, then submit that folder's view-only/public link.